GazeSync: Reproducible Synchronization of Screen Interaction and Multi-Source Eye Tracking

Yaxiong Lei*
University of St Andrews and University of Essex
St Andrews / Colchester
United Kingdom

Rishab Talwar
University of St Andrews
St Andrews, United Kingdom

Shijing He
King's College London
London, United Kingdom

Xinya Gong
University of St Andrews
St Andrews, United Kingdom

Yuheng Wang
University of St Andrews
St Andrews, United Kingdom

Xudong Cai
King's College London
London, United Kingdom

Zhongliang Guo
University of St Andrews
St Andrews, United Kingdom

Juan Ye
University of St Andrews
St Andrews, United Kingdom

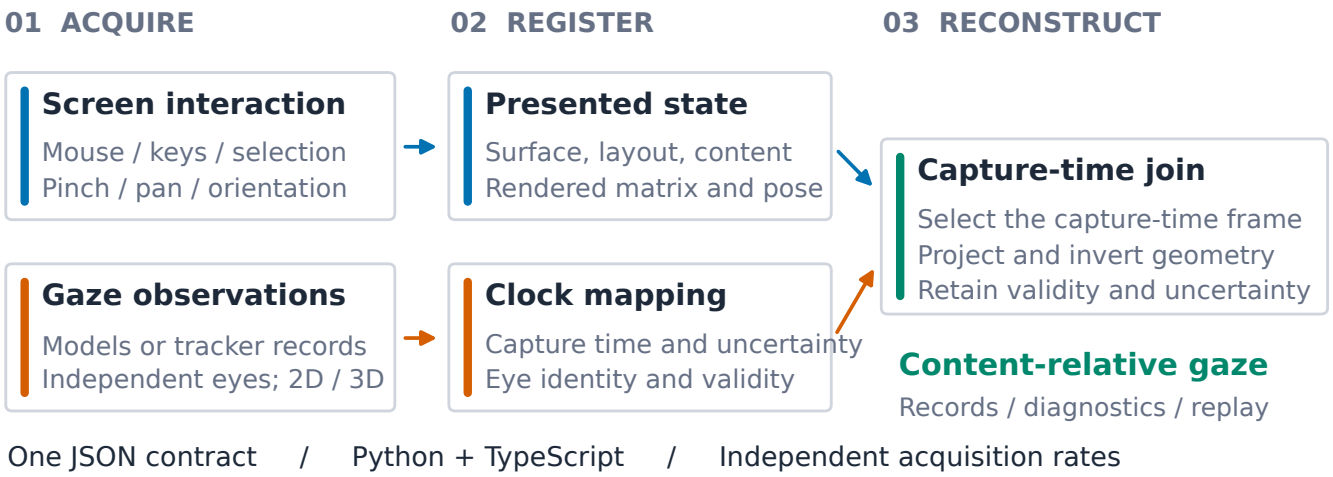


Figure 1: GazeSync separates screen presentation and eye-tracking acquisition until capture-time reconstruction. Input commands describe intent; the host records the view actually rendered. Each eye retains its timestamp, coordinate convention and validity. The same recording contract drives Python and TypeScript.

Abstract

A gaze sample and a zoom event can arrive together even though they describe different screen states. Recovering attention on manipulated content therefore requires explicit temporal and spatial provenance. We extend GazeSync into a standalone Python and TypeScript framework for desktop and mobile interaction, 2D points of gaze, 3D eye-origin rays, and independent monocular or binocular streams. The framework separates capture, input, presentation and receipt time; joins each observation to a surface-specific presentation history; and preserves uncertainty, eye identity, geometry and missing data in a replayable record. SDK-independent adapters normalize model and tracker records without assuming equal acquisition rates or manufacturing confidence. A reproducible synthetic matrix spans 78 configurations and 107,070 observations, including 30–240 Hz displays, 30–2,000 Hz gaze and nine viewport sizes. Both implementations select the reference capture frame in every case; maximum cross-language viewport disagreement is 4.55×10^{-13} logical pixels. These are numerical consistency results, not physical tracking-accuracy estimates. We provide an inspectable synchronization contract, diagnostic guidance, simulation fixtures and isolated distribution tooling, and identify the device measurements needed before making end-to-end accuracy claims.

Keywords

eye tracking, temporal synchronization, pan and zoom, binocular gaze, reproducibility, research software

1 Introduction

A person zooms into a map landmark, drags the map, and then resets the view. A screen-coordinate gaze point can refer to three different content locations during that sequence. A delayed tracker callback adds another ambiguity: the view visible when the callback is handled may differ from the view at exposure. On a phone, simultaneous finger movement and changing display pose introduce further spatial dependencies. Researchers need to distinguish

*Working author list for review. Implementation report draft, 15 October 2026; not an accepted conference paper. Implementation arXiv submission pending.

these effects before interpreting a trace as attention to a particular feature.

The original GazeSync introduced mobile gaze and image-transform logging with content-relative reconstruction [1]. This report extends that system's implementation boundary. It defines how a renderer and heterogeneous gaze sources exchange timing, geometry and validity, then provides matching Python and TypeScript implementations. The target is instrumentation of an application owned by the researcher: the library does not observe arbitrary operating-system windows or infer unrecorded visual states.

Our contribution has three parts. First, a capture-time contract preserves the screen state and source provenance needed to reconstruct content-relative gaze. Second, independent eye channels, source adapters and actionable diagnostics make heterogeneous measurements explicit. Third, deterministic simulations, cross-language conformance and isolated distribution artifacts make the implementation inspectable and repeatable. The evidence is a software audit and synthetic validation; no new participant or physical-device study is claimed. The development repository is https://github.com/GazeLei/GazeSync.

2 Background and scope

2.1 Relationship to the original GazeSync

The prior system used a Flutter/iOS application with an on-device gaze SDK and local logging server. Its callback pairing and nearest-state merge used a 50 ms tolerance. A three-participant formative study illustrated both useful content-relative reconstruction and calibration or compound-transform limitations [1]. Those participants and outcomes belong to the prior paper; they are not additional validation of this implementation.

The extension replaces the platform-specific merging assumption with explicit presentation intervals, source clocks and quality decisions. It adds desktop interaction, independent eye identity, metric ray projection, reusable adapters and a common record format across languages. Table 1 states the boundary between capabilities and evidence. A source can be accepted at many frequencies without establishing that a particular device exposes accurate capture timestamps at those frequencies.

2.2 Interaction and synchronization systems

Gaze-supported pan and zoom has been investigated as an interaction technique, including combining gaze with other input modalities [2]. Our instrumentation observes manual interaction while retaining gaze for analysis; it does not assume that the gaze target is the pointer or pinch midpoint. A person may inspect a landmark while it moves, look at a stable screen location while dragging, or shift attention ahead of the next action. The simulation therefore exercises several attention policies instead of coupling the gaze position directly to the input device.

Lab Streaming Layer provides timestamped multimodal streams and network clock synchronization [3]. Such infrastructure can supply a transport or clock estimate to a GazeSync host. Our core instead specifies which rendered content state a mapped timestamp refers to, how coordinates are reconstructed, and what makes the result ambiguous. It neither implements a network clock estimator nor claims to replace multimodal acquisition infrastructure. Device-specific data contracts also remain consequential: Tobii display coordinates and metric user coordinates differ from Pupil scene-camera coordinates [4, 5]. Explicit normalization is therefore part of the measurement contract, not a cosmetic import step.

Table 1: Implemented scope and evidence. "Supported" denotes a software contract, not certification of a device or browser.

Layer	Capability and evidence
Interaction	Desktop and mobile controller; deterministic gesture regressions and browser adapter tests.
Measurements	2D PoG, 3D rays, direction-only records; independent and combined eyes with synthetic fixtures.
Acquisition	Model and tracker record normalizers; SDKs, transports and physical calibration supplied by the host.
Synchronization	Clock mapping, presentation history and uncertainty flags; known-clock synthetic experiments.
Distribution	Python wheel/sdist and typed npm package; local installation checks, publication disabled.

3 Design requirements

Record what changed on screen. Mouse, keyboard and touch events express intent. A renderer can coalesce many inputs into one display update or animate a single command across several frames. We record input events for explanation and presentation snapshots for reconstruction. A snapshot includes surface, layout epoch, content identity, viewport dimensions, the rendered affine matrix, timing provenance and optional registered display geometry. Layout changes and suspension are observable events.

Respect independent measurements. A binocular tracker can lose one eye, sample eyes at different times, or expose a fused output with different semantics. The record retains `left`, `right`, `combined` or `unknown` identity. Separate observations retain their timestamps and validity. The software does not synthesize a second eye from a combined estimate or treat vendor validity as a probability.

Keep disagreement visible. Missing history, uncertain clocks, incompatible coordinates and invalid samples must remain inspectable. A seemingly smooth trace can conceal an acquisition error. Raw data and quality flags are retained alongside derived coordinates; blinks and gaps break a plotted trace. Rates and thresholds are configurable study parameters rather than universal recommendations.

Keep adaptation local and testable. The synchronization core depends on neither a hardware SDK nor a learned model. Adapters convert explicit records at the edge, while a single finite-JSON contract carries the result. This permits hardware or inference changes without changing the content-reconstruction rules. Cross-language fixtures check that Python analysis and TypeScript acquisition interpret the same record consistently.

4 Temporal reconstruction

4.1 Four event times and a clock epoch

We distinguish input time t_i, presentation time t_p, source capture time t_s and session receipt time t_r. A frozen mapping for clock epoch e converts source time to session capture time:

$$t_c = b_e + r_e(t_s - a_e), \tag{1}$$

where a_e and b_e are corresponding anchors and r_e is the clock-rate ratio. Canonical times are monotonic milliseconds. Browser timing follows the monotonic clock model of High Resolution Time [7]; an epoch wall clock cannot be substituted without a measured mapping.

The host supplies clock uncertainty and optional drift uncertainty. The latter grows with distance from the anchor; conversion also includes a conservative floating-point term. A revised fit or source restart requires a new epoch. These are supplied engineering bounds, not a fitted statistical confidence interval. The framework does not estimate clock parameters from network arrival or assume symmetric one-way transport delay. Receipt-only sources remain recordable but are excluded from capture-time attention analysis.

4.2 Presentation intervals and ordering

For the surface-specific history $\mathcal{H}_s$, alignment selects

$$j^* = \underset{j \in \mathcal{H}_s:\, t_j^p \le t_c}{\arg\max}\ (t_j^p, \text{sequence}_j). \tag{2}$$

The last submitted snapshot wins a timestamp tie. Each snapshot persists until its successor. Geometry is not interpolated through an unobserved animation, reset or content replacement. The host must record every relevant intermediate rendered state, or explicitly assert change-only presentation logging.

A call to `flush(h)` is a watermark: it declares that all presentations through h have already been submitted. Insertion at or before that watermark is rejected. Samples normally wait for the greater of the configured reorder window and maximum accepted clock uncertainty. Ready samples are sorted by capture time within each flush. Later arrivals remain identifiable even when retained history permits valid offline reconstruction. Impossible future times and excessive uncertainty are quarantined promptly, preventing indefinite pending-buffer growth.

The interval $[t_c - u_c, t_c + u_c]$ is checked against visual changes. If it crosses a relevant change, the output is marked ambiguous. Boundaries that repeat identical geometry do not create false ambiguity. Histories are indexed per surface; retention preserves the newest state of each active surface, so frequent updates on one display do not evict another display's only state. A required older state may still expire, producing an explicit history flag.

4.3 Why timing errors become spatial errors

For a viewport observation p_v and rendered matrix $M(t)$, content gaze is $M(t)^{-1}\widetilde{p}_v$. Away from discontinuities, a small timing displacement Δt yields a first-order spatial error proportional to

$$\left\| \frac{dM(t)^{-1}}{dt} \widetilde{p}_v \right\| |\Delta t|. \tag{3}$$

Fast zoom or translation can therefore amplify a modest delivery delay. At a reset or layout discontinuity this derivative approximation does not apply; the correct presentation interval must be selected explicitly. Figure 2 illustrates this mechanism with known synthetic timing. The receipt-time baseline is an intentionally incorrect association policy, not a benchmark against every strategy used by existing systems.

5 Spatial reconstruction and eye identity

5.1 Two-dimensional points and content transforms

A PoG declares viewport, normalized-viewport or normalized-display coordinates. The full-display case requires recorded display dimensions and viewport offsets in compatible units. Device pixel ratio is retained as metadata and is not an instruction to multiply logical coordinates again. The renderer supplies

$$M = \begin{bmatrix} a & c & e \\ b & d & f \\ 0 & 0 & 1 \end{bmatrix}, \qquad \widetilde{p}_c = M^{-1}\widetilde{p}_v. \tag{4}$$

This representation includes letterboxing, intrinsic image scaling, pivot translation and rotation without applying an offset twice. Coordinates outside the viewport or content remain visible and flagged rather than being clipped to an apparently valid target.

Each accepted frame caches its inverse. The inversion checks conditioning after normalizing the linear block by its largest absolute coefficient. It rejects singular or non-finite results while permitting uniformly small, well-conditioned transforms. This matters for large content coordinate systems whose displayed scale can be much less than one.

5.2 Metric rays and a registered display

A ray supplies origin O in metres, direction D and a frame identifier. The registered display supplies top-left point P, orthonormal right/-down axes u, v and metric dimensions w_m, h_m. After normalizing D, the forward crossing is

$$n = u \times v, \quad \lambda = \frac{(P - O) \cdot n}{D \cdot n}, \quad Q = O + \lambda D. \tag{5}$$

Zero, nearly parallel, backwards or incompatible rays do not fabricate a hit. For viewport size $W \times H$,

$$p_v = \left(W \frac{(Q - P) \cdot u}{w_m},\ H \frac{(Q - P) \cdot v}{h_m} \right). \tag{6}$$

The same content inverse then applies. This is a crossing of a registered display plane, not an inferred scene fixation or vergence depth.

Moving displays require pose timestamps and measured extrinsics. The current implementation uses the selected frame's recorded pose and checks its age; it does not interpolate unobserved sub-frame motion. A direction-only model output is retained as `direction3d` but cannot be projected by inventing an eye origin. Figure 3 makes these dependencies explicit.

5.3 Observation identity and optional pairing

Observation identity includes source, clock ID and epoch, eye, representation and sample ID. The same physical capture can thus

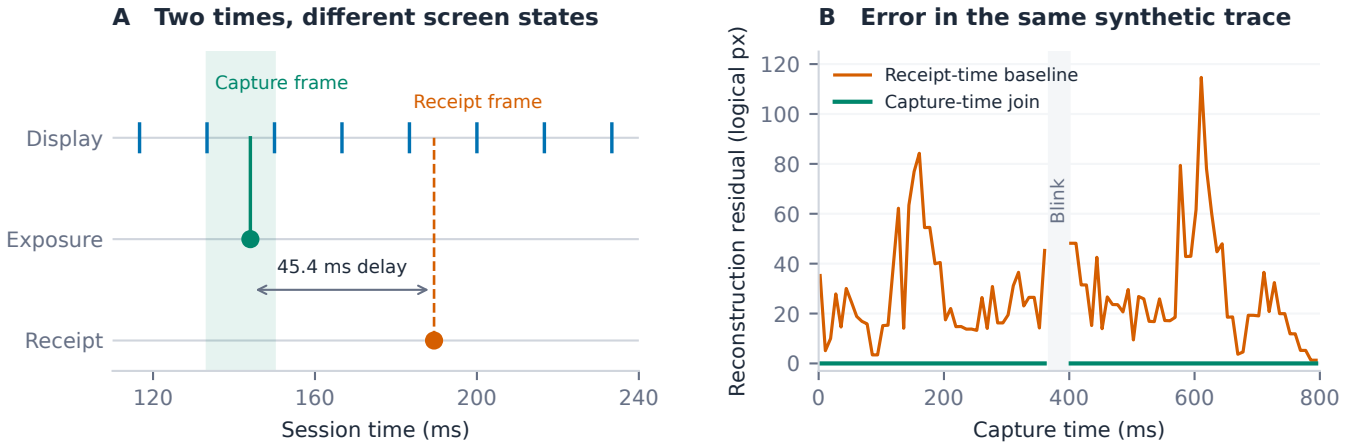


Figure 2: A representative synthetic trace at 60 Hz display, 240 Hz input and 120 Hz gaze per eye. (A) The same sample selects different display intervals at exposure and receipt. The shaded interval contains the true capture time. (B) Using receipt time produces content-reconstruction error during view change; capture-time residuals remain at numerical scale. The blink interval is missing data. Both panels are generated from saved simulation outputs, not a physical timing measurement.

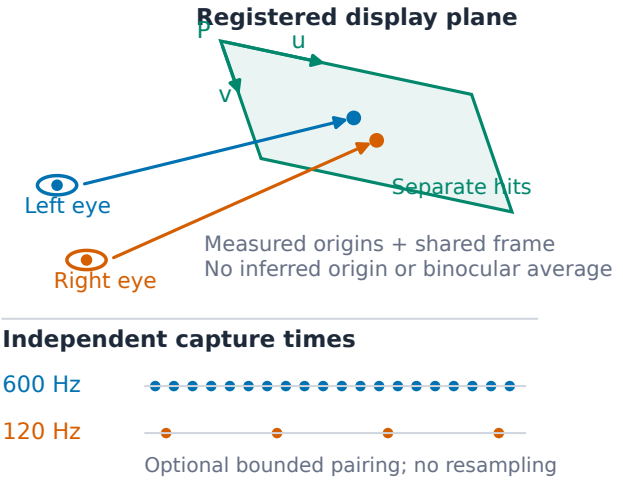


Figure 3: Independent eyes retain separate origins, rays, timestamps and validity. The display plane and both rays must share a measured coordinate frame. A schematic 600/120 Hz timeline illustrates unequal acquisition; optional bounded pairing neither resamples nor averages the eyes. Geometry is illustrative and not to scale.

carry left/right and PoG/ray records without collision. Optional capture and calibration identifiers retain further provenance. A combined estimate stays combined; no post hoc label turns it into an independent monocular observation.

For analyses that need matched eyes, an offline helper performs chronological, one-to-one pairing under a caller-specified maximum skew, defaulting to 5 ms. It considers only usable observations with matching source, representation, clock identity/epoch, surface/epoch and calibration. Unmatched observations are returned explicitly. Different source clocks can be synchronized by the host, but the helper conservatively leaves them separate. It does not implement an optimal assignment, estimate vergence or fuse measurements. Study-specific fusion requires its own uncertainty and validity policy.

6 Adapters, diagnostics and developer contract

6.1 SDK-independent normalization

Both language implementations expose generic model, Tobii Pro dictionary, EyeLink bridge and Pupil Core adapters. A generic model adapter supports nested field paths, explicit timestamp units, display-pixel normalization and metric origin conversion. Model inference and tensor transfer remain outside the core. A new adapter can be a pure function producing the shared JSON schema and structured warnings.

Tobii normalization preserves per-eye validity and normalized display points; optional rays are defined from the vendor origin to the reported display point, with millimetres converted to metres [4]. Confidence remains null when only validity is provided. The EyeLink bridge consumes serialized per-eye pixel records and millisecond device timestamps; a Python helper extracts this bridge record from a newly received PyLink sample [6]. Neither path bundles or licenses the hardware SDK on the user's behalf.

Pupil normalization preserves individual base-sample timestamps and deduplicates reused eye observations within a bounded history. Scene-camera coordinates do not become screen PoG without an explicit calibrated homography or registered surface record. Right/left IDs, bottom-origin coordinates and millimetre eye centres follow the vendor contract [5]. Mapping timestamp skew is retained and checked. These behaviors are tested with synthetic protocol fixtures; the release does not claim physical tracker validation.

6.2 Diagnostics as integration feedback

A source descriptor declares adapter, eye set, tracking mode, expected cadence and timestamp provenance. Expected cadence does not resample observations. Diagnostics estimate rates per source, eye, representation and clock epoch using bounded windows. They expose gaps, out-of-order delivery, rate deviations, missing history, uncertain clocks and unavailable geometry. Observers receive the first occurrence for each diagnostic context while the report retains counts and most recent occurrence times. An observer exception is recorded without interrupting acquisition.

A per-observation `usable` flag and reason list remain the primary quality contract. Informational conditions, such as unavailable scalar confidence, can coexist with usable geometry. Receipt-only timestamps, blinks, missing history or ambiguous presentation make content attention unusable. Raw measurements remain available for a different downstream analysis that states its own policy.

6.3 Human interaction and host responsibilities

The controller anchors wheel zoom at the pointer, distinguishes drag from selection-box zoom, supports keyboard reset and cancels an active gesture when another view command takes over. Touch interaction tracks contacts, pinch scale and midpoint translation; releasing one contact transitions to pan without a jump. Double tap, cancellation, resize, orientation change and background/resume are explicit cases. Browser handling follows pointer-event ownership and cancellation semantics [8]; it avoids capturing keys from embedded editable controls.

The browser adapter records application input, while the host records what was rendered. Native Python applications can feed the same controller logic from GUI callbacks. Both cores are synchronous; applications serialize rendering and SDK callbacks through a single session owner. No transport or thread scheduler is hidden in the measurement path. Host-specific kinetic scrolling or animation can be recorded as presentation snapshots without adding its physics to the core.

7 Verification and synthetic evaluation

7.1 Audit strategy and regression coverage

The audit follows failure paths across geometry, event ordering, browser input, recording, replay and package boundaries. Earlier defects included cross-surface frame selection, keyboard zoom overwritten by a stale drag baseline, unstable small-scale inversion, malformed-record mutation, delayed future-time quarantine and undetected replay gaps. Focused regression tests reproduce those triggers. The extension adds explicit eye identity, shared adapter fixtures, receipt-time exclusion, nullable confidence and clock-boundary checks in both languages.

Strict TypeScript compilation generates public declarations from the executable source. Consumer tests check exports and validity-dependent measurement types. Python tests exercise the independent implementation and validate normalized fixtures against JSON Schema. Release checks inspect imports, package contents, license consistency and shared fixtures. These tests establish specified behaviors under examined inputs; they are not a proof of general correctness.

7.2 Two complementary simulation families

Interaction scenarios. Desktop and mobile scenarios each last nine seconds at 60 Hz display and 120 Hz gaze, repeated with seeds 1, 41 and 90210. Desktop actions include wheel zoom, pan, marquee selection, cancellation, reset and resize. Mobile actions include pinch in/out, moving midpoint, pinch-to-pan transition, double tap, orientation and suspension. Attention policies include content-landmark tracking, screen-fixed attention during pan, scanning and reading-like fixation steps. Timing and motion parameters are illustrative test assumptions, not a fitted model of human eye movements. The moving-phone fixture holds pose between recorded frames and does not validate real continuous motion.

Rate and context matrix. A second family separates frequency sensitivity from context coverage. The controlled matrix contains 42 cells: six display rates (30, 60, 90, 120, 144 and 240 Hz) crossed with seven gaze rates per eye (30, 60, 120, 250, 600, 1,000 and 2,000 Hz). It fixes a 1280×720 viewport, 240 Hz input and independent binocular acquisition. A further 36 cases cross nine viewport configurations with left monocular, right monocular, independent binocular and combined modes. These context cases vary input rate, device pixel ratio, orientation and clock configuration together; they are coverage cases, not a factorial causal study.

The nine initial viewports range from 320×568 to 3840×2160, including portrait, landscape, tablet and ultrawide layouts. Context input rates range from 30 to 1,000 Hz. Independent context eyes use 600 and 120 Hz and separate clock mappings. Each case lasts 800 ms with seed 41. Inputs are coalesced into rendered frames. Ordinary delivery delay is 28–63 ms, with an additional 120 ms on selected captures. A blink interval occupies 46–50% of the trace. Uniform per-axis perturbation of ±0.35 logical pixels supplies an explicitly high-accuracy synthetic source; it is not a camera-model accuracy estimate.

PoG and ray observations originate from the same perturbed viewport target. Their agreement tests representation consistency rather than agreement between independent sensors. Reconstruction error is measured against that perturbed target, using the known renderer coefficients, and expressed back in logical viewport pixels. Sensor noise is therefore distinguished from software reconstruction residual. No human data are used in either simulation family.

7.3 Cross-language conformance and results

The test driver serializes identical events for both implementations and compares all aligned records and diagnostic reports. Keys, labels, frame identities and flags must match exactly. Numeric comparison uses absolute tolerance 10^{-8} plus relative tolerance 10^{-12}; the largest observed differences are much smaller (Table 2). The suite additionally compares ten source adapter fixtures and checks exact replay of derived alignment within the same implementation. Python-native experiment generation is exercised separately.

Across the 78 cases, all 107,070 expected observations are received, pending buffers drain, and every capture-frame selection matches the independent synthetic reference. Maximum reconstruction residual is 7.85×10^{-13} logical pixels. Maximum Python–TypeScript viewport disagreement is 4.55×10^{-13} pixels; mapped

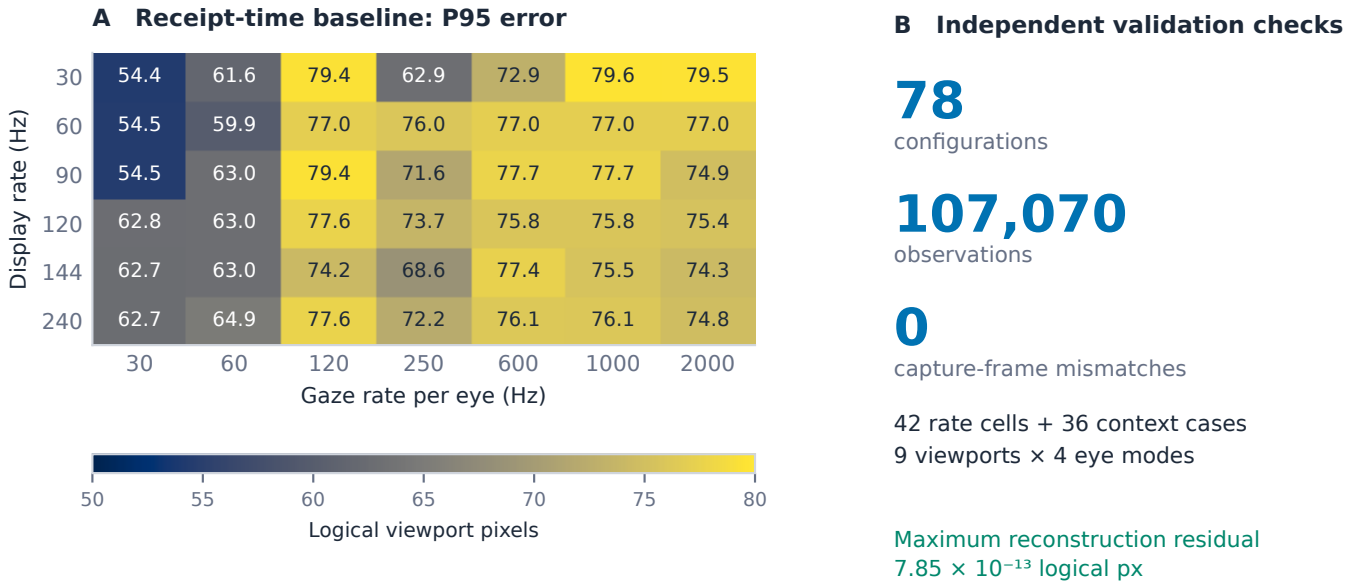


Figure 4: Controlled rate experiment and overall software checks. (A) P95 residual when deliberately using the receipt-time frame, at fixed 1280 × 720, 240 Hz input and independent binocular gaze; each cell is one seeded 800 ms trace. Values are descriptive, without participant-level inference. (B) The 42 rate cells and 36 additional context cases yield 107,070 observations. Correct capture-time reconstruction has zero frame mismatches and numerical-scale residuals. The extra context cases are reported separately because viewport, rate and orientation vary together.

Table 2: Numerical consistency on the 78-case matrix. Units distinguish content coordinates from logical viewport pixels.

Measure	Observed result
Expected / received observations	107,070 / 107,070
Capture-frame mismatches	0
Maximum reconstruction residual (px)	7.85×10^{-13}
Maximum cross-language viewport difference (px)	4.55×10^{-13}
Maximum cross-language content difference (units)	4.55×10^{-13}
Maximum capture-time difference (ms)	0
Exact within-implementation replay	78 / 78 cases
Pending observations after final drain	0

capture times agree at the reported precision. These near-zero values measure arithmetic and contract consistency for known inputs. They cannot establish subpixel physical accuracy.

The receipt-time baseline's P95 residual ranges from 54.4 to 79.6 logical pixels across the controlled cells (Figure 4A). This range belongs to the specified fast manipulation trace, viewport and delays. It is not a general error bound or a comparison between tracker brands. Increasing gaze rate cannot repair an incorrect temporal association. Context-case baseline values are not pooled to infer frequency effects because dimensions and motions also differ.

8 Reproducibility and distribution

The repository contains strict TypeScript source, Python source, JSON schemas, synthetic fixtures, regression tests, a local interaction lab and figure scripts. The two libraries have no runtime dependencies. npm ships executable modules, generated declarations and corresponding TypeScript source. Python ships a wheel and source distribution; SDKs, inference weights and private-project data are excluded. Clean installation tests run outside the checkout, including a wheel rebuilt from the source distribution.

Schema 2.0 records the implementation version, source descriptors, clock mappings, configuration, raw event sequence, retention losses and derived output. Exact replay rejects incomplete or incompatible logs. Default in-memory limits are explicit: 7,200 frames, 100,000 raw/derived records and 4,096 pending observations. Long sessions require persistence before rollover. Current bounded arrays still incur front-removal costs; streaming storage and backpressure remain measured integration work rather than claimed features.

The full matrix can be reproduced with the following source-checkout commands:

```
npm run test:conformance
node tools/paper-data.mjs
.venv/bin/python tools/paper_figures.py
```

Figure generation additionally requires the pinned plotting dependencies listed in the paper build guide. The reported local run used Node.js 26.7.0 and Python 3.12.14. Continuous integration is configured across Node 22/24/26 and Python 3.10/3.12/3.14. The aggregate JSON data preserve every configuration and result; generated working sessions remain excluded from source control.

Code is licensed GPL-3.0-only. The manuscript has separate draft publication rights. The repository is currently private and registry publishing is disabled; no public package availability or implementation arXiv identifier is claimed. Maintainer-reviewed workflows prepare the official npm and PyPI routes. Citation metadata requests both the original GazeSync paper and this implementation report; it will be updated with the real preprint identifier after submission.

9 Limitations and evaluation agenda

The largest uncertainty lies outside the synthetic equations: physical exposure time, display scanout, clock offset and drift, extrinsic calibration, source accuracy and moving-device pose. Render callbacks alone do not measure photon onset. Replaying known geometry cannot validate an assumed camera-to-display transform. Protocol fixtures cannot establish behavior across firmware or SDK versions, and a phone-sized browser simulation cannot certify touch behavior on every mobile browser.

The next evaluation should measure timing against an external reference, report latency distributions and uncertainty, and test display geometry on held-out targets. Static views, pan, zoom, rotation and device motion should be assessed separately before compound manipulations. Relevant outcomes include temporal ambiguity, usable coverage, per-eye dropout, viewport/content error and angular ray error. Calibration and assessment targets must remain separate. Repeated sessions and participant-separated analysis are needed to characterize drift and generalization.

Human behavior also needs empirical grounding. The scripted attention policies exercise distinct reconstruction cases but do not estimate how often people follow a pointer, retain a landmark or look ahead of an action. A future study should record task intent and interaction phases alongside independent gaze, then test whether the instrumentation preserves meaningful content attention without changing normal interaction. Longer runs should measure memory, persistence overhead and worst-case latency under realistic acquisition load.

10 Conclusion

GazeSync provides an explicit bridge between heterogeneous eye-tracking records and the content state visible during desktop or mobile interaction. Its temporal contract, geometric conventions, independent eye records and quality flags make assumptions inspectable. Matching Python and TypeScript implementations, adapter fixtures and synthetic experiments establish a reproducible software foundation. Measured device timing, calibration and participant behavior remain the necessary next evidence for end-to-end claims.

References

[1] Yaxiong Lei, Rishab Talwar, Shijing He, Xinya Gong, Yuheng Wang, Xudong Cai, Zhongliang Guo, and Juan Ye. 2026. GazeSync: A Mobile Eye-Tracking Tool for Analyzing Visual Attention on Dynamically Manipulated Content. In *Extended Abstracts of the CHI Conference on Human Factors in Computing Systems*. https://doi.org/10.1145/3772363.3798725

[2] Sophie Stellmach and Raimund Dachselt. 2012. Investigating Gaze-supported Multimodal Pan and Zoom. In *Proceedings of the Symposium on Eye Tracking Research and Applications (ETRA)*. https://imld.de/cnt/uploads/2013/07/2012-ETRA-GazeSuppPanAndZoom.pdf

[3] Christian Kothe, Seyed Yahya Shirazi, Tristan Stenner, David Medine, Chadwick Boulay, Matthew I. Grivich, Fiorenzo Artoni, Tim Mullen, Arnaud Delorme, and Scott Makeig. 2025. The lab streaming layer for synchronized multimodal recording. *Imaging Neuroscience* 3. https://doi.org/10.1162/IMAG.a.136

[4] Tobii Pro. Coordinate systems and Python SDK documentation. https://developer.tobiipro.com/commonconcepts/coordinatesystems.html. Accessed 15 October 2026.

[5] Pupil Labs. Pupil Core developer documentation. https://docs.pupil-labs.com/core/developer/. Accessed 15 October 2026.

[6] SR Research. Python programming with EyeLink. https://www.sr-research.com/support/thread-48-post-47.html. Accessed 15 October 2026.

[7] W3C. High Resolution Time, Level 3. https://www.w3.org/TR/hr-time-3/. Accessed 15 October 2026.

[8] W3C. Pointer Events, Level 3. https://www.w3.org/TR/pointerevents3/. Accessed 15 October 2026.